

1.1 Process Concurrency & Race Conditions

- **Independent Process:** State not shared. Deterministic execution. Never causes data corruption.
- **Cooperating Process:** Shares variables, memory, buffer, or files. Non-deterministic relative sequence.
- **Race Condition:** Concurrent access to shared data without synchronization where final result depends on relative execution interleaving.
- **Lost Update Assembly Trace:** Balance = 1000. P1 deposits 500 (R1=1500). Preempted before store. P2 withdraws 200 (R2=800, stores 800). P1 wakes and stores 1500. Outcome: Corrupt balance (1500 or 800 instead of 1300)!

1.2 Critical Section Problem & The 4 Criteria

- **4 Code Sections:** **Entry Section** (Acquires lock) $\rightarrow$ **Critical Section** (Shared resource access; only 1 process) $\rightarrow$ **Exit Section** (Releases lock/signals) $\rightarrow$ **Remainder Section** (Local code).

Criterion	Type	Definition & Invariant	Hazard Prevented
Mutual Exclusion	Primary (Mandatory)	If P_i is in CS, no other process can be in CS. Zero concurrent overlap.	Data Inconsistency
Progress	Primary (Mandatory)	If CS is empty, only processes not in remainder decide next entrant. Decision not delayed indefinitely.	Deadlock / Livelock
Bounded Waiting	Secondary (Desirable)	Bound k on entries granted to others after a request is registered.	Starvation
Speed Neutrality	Secondary (Desirable)	No assumptions regarding relative CPU clock speeds or processor counts.	Architecture Bugs

1.3 Software Solutions: Peterson's Algorithm (Gary Peterson, 1981)

Classical 2-Process software synchronization satisfying Mutual Exclusion, Progress, and Bounded Waiting ($k=1$).

Process P0 Code (i=0, j=1)

```
flag[0] = true; // Declare intent
turn = 1;      // Polite tie-breaker
while (flag[1] && turn == 1); // Spin
/* CRITICAL SECTION */
flag[0] = false; // Release intent
```

Process P1 Code (i=1, j=0)

```
flag[1] = true; // Declare intent
turn = 0;      // Polite tie-breaker
while (flag[0] && turn == 0); // Spin
/* CRITICAL SECTION */
flag[1] = false; // Release intent
```

- **Proof of Mutual Exclusion:** For both to be in CS, $turn == 0$ and $turn == 1$ must hold simultaneously in RAM, which is impossible for a single integer variable!
- **Proof of Progress & Bounded Waiting:** If P1 is in Remainder, $flag[1] == false$, allowing P0 immediate entry. When exiting, P0 sets $turn = 1$, giving P1 entry within $k=1$ turn.

1.4 Software Solutions: Dekker's Algorithm (Theodorus Dekker, 1965)

First correct software mutual exclusion solution. Uses $flag[2]$ and $turn$ with a contention back-off loop.

```
flag[i] = true;
while (flag[j]) {
    if (turn == j) {
        flag[i] = false; // Polite withdrawal of intent to prevent Livelock
        while (turn == j); // Wait until Pj completes CS
        flag[i] = true;   // Re-assert intent
    }
}
/* CRITICAL SECTION */
turn = j; flag[i] = false;
```

Dekker vs Peterson: Dekker requires nested checks and active intention withdrawal; Peterson simplified this to a single while condition without withdraw.

2.1 Hardware Synchronization Primitives (Test-and-Set & Swap)

- **Software Lock Flaw:** `while(lock==1); lock=1;` fails because preemption between test and set allows two processes into CS simultaneously!
- **Test_and_Set Instruction (TSL):** Hardware atomic CPU instruction executed in 1 clock cycle:

```
boolean Test_and_Set(boolean *target) { boolean r = *target; *target = true; return r; }
// Synchronization Code:
while (Test_and_Set(&lock)); /* CRITICAL SECTION */ lock = false;
```

- **Swap Instruction:** Atomically swaps memory contents: `while(key) Swap(&lock, &key);`
- **Spinlocks:** Busy-waiting locks. Excellent for short waits (avoids context switch overhead); terrible for long waits (100% CPU burn). Guarantees Mutual Exclusion and Progress, but NOT Bounded Waiting (Starvation prone).

2.2 Semaphores (Edsger Dijkstra, 1965)

Protected integer variable modified strictly via atomic primitives `wait()` (P / Down) and `signal()` (V / Up).

Type	Value Range	Primary Purpose	Initialization
Binary Semaphore	0 or 1	Mutual Exclusion (Critical Section Lock).	1 (Unlocked)
Counting Semaphore	0 to N	Resource Pool Management (e.g. N printers, DB connections).	N (Available count)

Block & Wakeup Implementation (Eliminating Busy Waiting)

```
typedef struct { int value; struct process *list; } semaphore;
void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) { add_to_queue(S->list, current_process); block(); } // Puts to sleep!
}
void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) { process *P = remove_from_queue(S->list); wakeup(P); } // Wakes to Ready!
}
```

Magic Rule: If `S.value < 0`, magnitude `|S.value|` equals exact number of processes blocked in the queue!

2.3 Mutex vs Binary Semaphore Comparison

Feature	Mutex (Mutual Exclusion Lock)	Binary Semaphore
Mechanism	Strict Locking mechanism.	Signaling / Notification mechanism.
Ownership	Has ownership: Only locking thread may unlock.	No ownership: Any process can signal.
Priority Inversion	Supports Priority Inheritance Protocol (PIP).	No priority inheritance support.

2.4 Producer-Consumer (Bounded Buffer) Problem

Shared circular buffer of size N . 3 Semaphores: `mutex = 1`, `empty = N`, `full = 0`.

Producer Process

```
wait(empty); // Decrement empty
wait(mutex); // Lock buffer
buffer[in] = item;
in = (in + 1) % N;
signal(mutex);
signal(full); // Increment full
```

Consumer Process

```
wait(full); // Decrement full
wait(mutex); // Lock buffer
item = buffer[out];
out = (out + 1) % N;
signal(mutex);
signal(empty); // Increment empty
```

⚠ **Deadlock Trap:** If Producer executes `wait(mutex)` before `wait(empty)`, when the buffer is full, it locks mutex and sleeps on empty. Consumer cannot enter to consume $\rightarrow$ DEADLOCK!

3.1 Readers-Writers Problem (First Problem: Reader Priority)

- **Conflict Matrix:** Readers concurrent (✓); Reader & Writer (✗); Multiple Writers (✗).
- **Variables:** `int rc = 0; semaphore mutex = 1; semaphore db = 1;`
- **Reader Protocol:** `wait(mutex); rc++; if (rc == 1) wait(db); signal(mutex);` → Read DB → `wait(mutex); rc--; if (rc == 0) signal(db); signal(mutex);`
- **Writer Protocol:** `wait(db);` → Write DB → `signal(db);`
- **Writer Starvation:** If readers stream continuously, `rc` never drops to 0 → Writer starves indefinitely!

3.2 Dining Philosophers & Sleeping Barber Problems

- **Dining Philosophers:** 5 philosophers, 5 chopsticks. All pick left chopstick simultaneously → Circular Wait → Deadlock!
Deadlock Solutions: (1) Allow at most 4 diners. (2) Asymmetric pick: Odd pick left first, Even pick right first.
- **Sleeping Barber:** 1 barber, 1 barber chair, `N` waiting chairs.
Semaphores: `customers = 0, barbers = 0, mutex = 1, waiting = 0.`
If waiting chairs < `N`, customer increments waiting and signals customer; otherwise leaves (Dropout).

3.3 Process Generation, Lifecycle & Forking Mathematics

- **The fork() Return Values:** In Child: `0` | In Parent: `Child PID (>0)` | Error: `-1`.
- **Process Tree Growth Formulas:** For `n` consecutive `fork()` calls:
$$2^n \times \text{Total Processes} = 2^n \times \text{New Child Processes Created} = 2^n - 1$$
Example: 3 forks → $2^3 = 8$ total processes, 7 child processes.
- **Process State Duality:**
 - **Zombie** (`<defunct>`): Child dead, Parent has NOT called `wait()`. Entry wastes PID in Process Table.
 - **Orphan:** Parent died before child. Child adopted by `init` / `systemd` (PID 1).

3.4 Inter-Process Communication (IPC) Models

Feature	Shared Memory Model	Message Passing Model
Speed	Fastest (Direct memory read/write at RAM bus speed).	Slower (Syscalls + kernel data copying).
Synchronization	Programmer responsibility (via Semaphores).	Kernel handles synchronization automatically.
Primitives	<code>shmget()</code> , <code>shmat()</code>	<code>send(message)</code> , <code>receive(message)</code>
Distributed Support	Single machine only.	Works seamlessly across networked systems.

- **IPC Schemes:** Direct (`send(P, msg)`) vs Indirect (`send(Mailbox_A, msg)`).
- **Buffering:** Zero Capacity (Rendezvous / Synchronous), Bounded Capacity (`N`), Unbounded Capacity.